

Employee Demo Program

Aim/Objective:

To develop a Java program to demonstrate constructor overloading by creating Employee objects using a default constructor, a constructor with two parameters, and a constructor with all parameters.

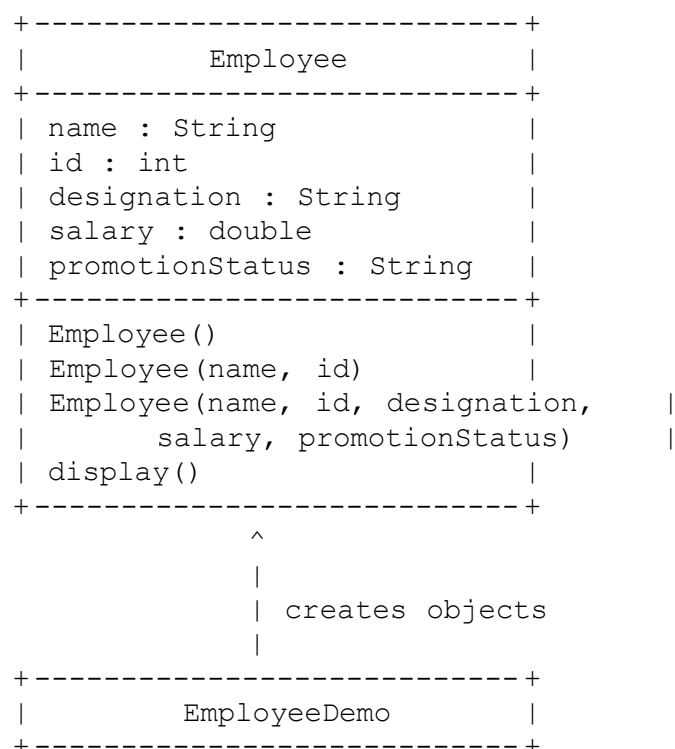
System Requirements:

- Operating System: Windows or any operating system supporting Java.
- Programming Language: Java.
- Java Development Kit (JDK).
- Text Editor or IDE: VS Code / Eclipse / IntelliJ IDEA.
- Command Prompt or Terminal.

Algorithm/Logic Description:

1. Start.
2. Create a class named Employee.
3. Declare name, id, designation, salary, and promotionStatus as data members.
4. Create a default constructor and initialize the employee with default values.
5. Create a constructor with name and id as parameters.
6. Create another constructor with all employee details as parameters.
7. Create a display() method to print the employee details.
8. Create object e1 using the default constructor.
9. Create object e2 using the two-parameter constructor.
10. Create object e3 using the constructor with all parameters.
11. Display the details of all three employees.
12. Stop.

UML Class Diagram:



```
| main(String[] args) |  
+-----+  
|
```

Source Code:

```
class Employee  
{ String  
    name;  
  
    int id;  
    String  
    designation;  
    double salary;  
    String promotionStatus;  
  
    // Default  
    Constructor  
    Employee() {  
        name = "Not  
        Assigned"; id =  
        0;  
        designation =  
        "Trainee"; salary  
        = 0;  
        promotionStatus =  
        "No";  
    }  
  
    // Constructor with two  
    parameters Employee(String name,  
    int id) {  
        this.name =  
        name; this.id =  
        id;  
        designation =  
        "Developer"; salary =  
        30000;  
        promotionStatus =  
        "No";  
    }  
  
    // Constructor with all  
    parameters Employee(String name,  
    int id,  
        String  
        designation,  
        double salary,  
        String promotionStatus) {  
  
        this.name =  
        name; this.id =  
        id;  
        this.designation =  
        designation; this.salary =  
        salary; this.promotionStatus  
        = promotionStatus;  
    }  
  
    void display() {  
        System.out.println("Name : " + name);  
        System.out.println("ID : " + id);  
        System.out.println("Designation : " +
```

```

        designation); System.out.println("Salary :
        " + salary);
        System.out.println("Promotion Status : " +
        promotionStatus); System.out.println();
    }
}

public class EmployeeDemo {
    public static void
        main(String[] args) {
        Employee e1 = new
        Employee(); Employee e2 =
        new Employee(
            "Meghana", 101);

        Employee e3 = new
            Employee(
                "Ravi", 102,
                "Manager",
                60000,
                "Yes");

        System.out.println("Employee
        1"); e1.display();

        System.out.println("Employee
        2"); e2.display();

        System.out.println("Employee
        3"); e3.display();
    }
}

```

Compilation and Execution Commands:

Save the file as: EmployeeDemo.java

Compile:

```
javac EmployeeDemo.java
```

Run:

```
java EmployeeDemo
```

Output Screenshot:

```
PS C:\Users\VYSHNAVI\OneDrive\Desktop\megh> javac EmployeeDemo.java
PS C:\Users\VYSHNAVI\OneDrive\Desktop\megh> java EmployeeDemo

Employee 1
Name : Not Assigned
ID : 0
Designation : Trainee
Salary : 0.0
Promotion Status : No

Employee 2
Name : Meghana
ID : 101
Designation : Developer
Salary : 30000.0
Promotion Status : No

Employee 3
Name : Mani
ID : 102
Designation : Manager
Salary : 60000.0
Promotion Status : Yes
```

Result:

Thus, the Java program was successfully executed and constructor overloading was demonstrated using the Employee class. The details of three employees were displayed using different constructors.